KUBERNETES COMPREHENSIVE · AXP-K8S-5D

Day 1 Foundations & First Deployment

Participant Manual

Platform	AxisPay (fictional)
Kubernetes	v1.36
Environment	Ubuntu 26.04 LTS · Minikube
Built	11 August 2026

Every merchant, customer, card token, acquirer and transaction in this manual is fictional. No real institution is represented and no card number exists anywhere in this platform — only tokens. The platform is **not** PCI-DSS compliant and is not presented as such; it uses PCI-shaped constraints to make security controls consequential in a training context.

Day 1 — Foundations and First Deployment

AxisPay · Kubernetes Comprehensive · Participant Manual, Chapter 1

How to read this chapter

Every topic below follows the same 16-point structure. That consistency is deliberate: once you know the shape, you can find "the security bit" or "the troubleshooting bit" for any topic without hunting.

The classroom gives you the mental model. **This manual gives you the depth** — the high availability, disaster recovery, performance and security material that would take three days to deliver as lecture. Read the relevant section the evening before you need it, and again a month after the course when you meet the topic in production.

Section	Answers
1. What it is	Plain English, no jargon in the first sentence
2. Why it exists	What was painful before
3. Business problem	In AxisPay terms, with money attached
4. How it works	The mechanism, in order
5. Internal architecture	Components and responsibilities
6. Component interactions	Who calls whom
7. Enterprise example	How a real payments platform uses it
8. Real-world analogy	One analogy — and its limits
9. Best practices	Rules, each with a reason
10. Common mistakes	With the symptom each produces
11. Security	Threat → control → residual risk
12. Performance	What gets slow, at what scale
13. High availability	Surviving node and zone loss
14. Disaster recovery	What to back up, RTO/RPO
15. Monitoring	Specific metrics and thresholds
16. Troubleshooting	Symptom → cause → command → fix
+	Interview questions with model answers

1.1 The declarative model and the reconciliation loop

1. What it is

You do not tell Kubernetes what to do. You tell it what you want, and it works continuously to make that true.

You write down a description of the world you want — "three copies of the payment service should be running" — and hand it to the cluster. Something inside the cluster then compares that description against what is actually running, notices any difference, and acts to close the gap. It does this forever, whether or not you are watching.

2. Why it exists

Before this model, operations was a sequence of instructions someone had to execute: *start this process on that server, and if it dies, start it again.* The instruction was only as reliable as the person or script running it, and the moment reality drifted from intention — a crash, a reboot, a full disk — nothing corrected it until a human noticed.

The declarative model moves the intention into the system itself. Correction becomes automatic because it is the system's only job.

3. The business problem

At 02:14 a node in AxisPay's cluster runs out of memory. The kernel's OOM-killer terminates the single `payment-service` process.

Without reconciliation: card authorisations fail for 41 minutes until an engineer wakes, reads the alert and restarts the process. Kalahari Coffee Roasters lose R38,000 of Saturday trade. Three merchants breach their contractual uptime SLA.

With reconciliation: a controller observes that 2 pods exist where 3 should, and creates a replacement. Elapsed time: about eight seconds. Nobody is paged. The engineer reads about it over breakfast.

4. How it works

Every object in Kubernetes has two halves:

Half	Meaning	Who writes it
`spec`	Desired state — what you want	**You**
`status`	Actual state — what is	A **controller**

A controller is a program that runs this loop, forever:

```
1. OBSERVE   read spec and status from the API server
2. DIFF      compare them
3. ACT       if they differ, take one step towards spec
4. REPEAT
```

That is the whole idea. Everything else in this course is a variation on it.

5. Internal architecture

Controllers do not poll. They use a **watch** — a long-lived streaming connection to the API server that pushes changes as they happen. Each controller maintains a local cache (an *informer*) and a work queue.

```
API server ──watch──▶ Informer ──▶ Work queue ──▶ Worker ──▶ API server
                      (cache)       (dedup,        (reconcile)   (write)
                                     rate limit)
```

Three properties matter in practice:

- **Level-triggered, not edge-triggered.** A controller acts on the *current state*, not on the event that woke it. If it misses an event, the next sync corrects anyway. This is why Kubernetes is resilient to controllers restarting.
- **Idempotent.** Reconciling twice produces the same result as reconciling once.
- **Eventually consistent.** There is always a window where `status` lags `spec`. That window is why `kubectl get` immediately after `kubectl apply` shows nothing yet.

6. Component interactions

Deleting one pod from a 3-replica Deployment:

```
you             kubectl delete pod
API server      pod marked for deletion, status written to etcd
ReplicaSet      watch fires → observes 2 running, spec says 3
controller      creates a new Pod object (spec.nodeName is EMPTY)
scheduler       watch fires → filters and scores nodes → binds pod to axispay-m02
kubelet(m02)    watch fires → pod assigned to me → calls containerd
containerd      pulls (cached) image, starts container
kubelet         writes status: Running back to the API server
```

Five components. None called another. Each watched the API server and did one small thing.

7. Enterprise example

A tier-1 bank runs 4,000 microservice replicas across three availability zones. Nodes are replaced continuously — patched, rotated for compliance, reclaimed by the cloud provider. On a normal day, several hundred pods are destroyed and recreated. No human is involved in any of it, and no ticket is raised. The reconciliation loop absorbs the churn.

The operational discipline that makes this safe is that every workload declares what it needs (replicas, resources, disruption budget) and the platform enforces it. Nobody logs into a node.

8. Real-world analogy

A thermostat. You set 21°C — that is `spec`. It reads the room temperature — that is `status`. If the room is 19°C it turns on the heating. It does not execute a script called "heat the room for twelve minutes"; it continuously compares and corrects.

Where the analogy breaks: a thermostat controls one variable with one actuator. Kubernetes runs hundreds of independent controllers over thousands of objects, and controllers can conflict — two controllers both trying to own the same pod will fight. That is why `spec.selector` overlap between Deployments is a genuine (and confusing) production bug.

9. Best practices

Practice	Reason
Keep manifests in Git and apply from there	The repository becomes the real source of truth; the cluster becomes a projection of it
Use `kubectl apply`, not `create`	`apply` is idempotent and records intent in an annotation, so re-running is always safe
Never edit live objects with `kubectl edit` in production	Your change is invisible to Git and will be silently reverted by the next apply
Treat `status` as read-only, always	Writing status is a controller's job; if you find yourself wanting to, you want a different object
Set `metadata.annotations["kubernetes.io/change-cause"]`	`kubectl rollout history` becomes readable during an incident
Expect eventual consistency	Poll with `kubectl rollout status` rather than asserting immediately after apply

10. Common mistakes

Mistake	Symptom
Using `kubectl scale` and forgetting the YAML	Replica count silently reverts on the next `apply`. "It worked yesterday."
Expecting `apply` to be synchronous	Script checks for pods immediately, finds none, and reports failure
Assuming deleting an object stops the controller	Deleting a *pod* does nothing lasting; you must delete the *controller*
Editing a field the controller owns	Your change is overwritten within seconds, apparently at random
Two Deployments with overlapping selectors	Pods flap between owners; replica counts oscillate. Very hard to diagnose.

11. Security considerations

Threat	Control	Residual risk
Anyone who can write `spec` controls the workload	RBAC on verbs (`create`, `update`, `patch`) per resource per namespace	A compromised CI service account can still deploy anything within its grant
A malicious manifest schedules a privileged pod	Pod Security Admission at namespace level (Day 5)	Admission runs at write time; existing pods are unaffected
Controllers hold broad permissions by design	Audit `ClusterRoleBinding`s; prefer namespaced `Role`s	The controller-manager itself is necessarily privileged
`kubectl edit` bypasses code review	Enforce GitOps; restrict `update` in production namespaces	Break-glass access must still exist for incidents

12. Performance considerations

- **Watch, not poll.** A controller that polls the API server every second across 10,000 objects will saturate it. All in-tree controllers use watches; if you write an Operator, use the informer framework rather than a loop.
- **Resync period.** Informers do a full resync periodically (commonly 10 minutes) to correct any missed events. Shorter resyncs increase API server load for little benefit.
- **etcd is the ceiling.** Every write goes through Raft consensus. Clusters degrade at roughly 5,000+ nodes or very high object churn; the symptom is rising `etcd_disk_wal_fsync_duration_seconds`.
- **Leading indicator:** `apiserver_request_duration_seconds` p99 climbing above ~1s for `LIST` operations means something is listing too much, too often.

13. High availability

The control plane should run three or five API server, scheduler and controller-manager instances across failure domains. Scheduler and controller-manager use **leader election** — only one is active; the others stand by and take over within seconds.

Critically: if the entire control plane is down, running workloads keep running. Kubelets continue managing their existing pods and containers keep serving traffic. What stops is *change*: no new pods, no rescheduling, no scaling, no self-healing. This distinction matters enormously during an incident — a control-plane outage is serious but is not, by itself, a customer-facing outage.

14. Disaster recovery

What to back up	etcd. It contains every object. Nothing else in the control plane holds state.
How	`etcdctl snapshot save` on a schedule, shipped off-cluster, encrypted at rest
RPO	Your snapshot interval. Hourly is common; 15 minutes for regulated workloads.
RTO	15–60 minutes to restore etcd and verify, on a practised team
The rule	A backup you have never restored is not a backup. Test the restore quarterly.
Better still	Keep every manifest in Git. Then etcd restore is a convenience, not a lifeline — you can rebuild from source.

15. Monitoring

Metric	Why	Alert at
`kube_deployment_status_replicas_available` vs `..._spec_replicas`	The gap the loop is failing to close	Gap persists > 5 min
`workqueue_depth` (per controller)	Controller falling behind	Sustained rise
`workqueue_adds_total` rate	Reconciliation churn — often a fight between controllers	Unexplained spike
`apiserver_request_duration_seconds` p99	Control-plane health	> 1s
`etcd_server_leader_changes_seen_total`	Control-plane instability	Any increase

16. Troubleshooting

Symptom	Likely cause	Command	Fix
Applied, nothing happened	Wrong namespace or context	`kubectl config view --minify`	Set context/ namespace
Object reverts after edit	A controller owns that field	`kubectl get <obj> -o yaml | grep -A5 ownerReferences`	Edit the owner, not the object
Replica count oscillates	Two controllers with overlapping selectors	`kubectl get deploy -o json | jq '.items[].spec.selector'`	Make selectors disjoint
`status` never updates	Controller unhealthy	`kubectl -n kube-system logs -l component=kube-controller-manager`	Restart or investigate
Everything stuck, nothing scheduling	Scheduler down or all nodes tainted	`kubectl get pods -n kube-system`; `kubectl describe node`	Restore scheduler; check taints

Interview questions

1. **What is the difference between imperative and declarative management, and why does it matter operationally?** *Imperative specifies the steps; declarative specifies the outcome and a controller determines the steps continuously. Operationally: declarative is idempotent, self-correcting, and reviewable in version control. The practical consequence is that drift is corrected automatically rather than discovered by a customer.*

2. **A pod belonging to a Deployment is deleted. Trace what happens.** *Actual state drops to N−1. The ReplicaSet controller observes the difference and creates a new Pod object with no `nodeName`. The scheduler filters and scores nodes and binds the pod. The kubelet on that node sees a pod assigned to it and instructs the container runtime to start it, then writes status back. Around eight seconds end-to-end.*

3. **The entire control plane is down. What still works?** *Existing pods keep running and serving traffic — kubelets do not need the control plane for steady state. What stops is change: no scheduling, no scaling, no self-healing, no `kubectl`. Serious, but not automatically a customer-facing outage.*

4. **Why are Kubernetes controllers level-triggered rather than edge-triggered?** *(senior) Level-triggered means acting on current state rather than on the event that triggered the wake-up. This makes the system resilient to*

lost events, controller restarts and network partitions — the next reconcile corrects regardless. Edge-triggered systems must guarantee event delivery, which is far harder to build correctly.

5. **You update a field on a live object and it reverts within seconds. Why?** *(senior) A controller owns that field and is reconciling towards its own* `spec`*. Check* `ownerReferences` *to find the owner and change it there. A common example is editing a pod that belongs to a ReplicaSet.*

1.2 Cluster architecture

1. What it is

A Kubernetes cluster is a set of machines split into two roles: a **control plane** that decides what should happen, and **nodes** that run the actual workloads.

2. Why it exists

Separating decision-making from execution allows either to fail independently and be scaled independently. The control plane can be made highly available without touching workloads; nodes can be added, drained and destroyed without the cluster losing its mind.

3. The business problem

AxisPay must survive losing a machine without losing payments. That requires something with a global view — which nodes exist, what is running, what should be — that is not itself running on the machine that failed.

4. How it works

The single most important structural fact: every component communicates only with the API server. The scheduler never calls the kubelet. The controller manager never calls etcd. Each component watches the API server for the objects it cares about and writes its results back.

This is what makes Kubernetes extensible: to add behaviour, you write another program that watches the API server. It is also why `kubectl` can observe everything — there is only one place anything happens.

5. Internal architecture

Control plane

Component	Single responsibility	Notes
kube-apiserver	The only door. Every request passes authentication → authorisation → admission.	Stateless; scale horizontally
etcd	The only source of truth. Distributed key-value store holding every object.	Raft consensus; needs odd member count
kube-scheduler	Decides **where**. Filters unsuitable nodes, scores the rest, binds the winner.	Writes only a node name. **Never starts anything.**
kube-controller-manager	Runs ~30 reconciliation loops — ReplicaSet, Deployment, Endpoint, Job, Node.	Leader-elected
cloud-controller-manager	Cloud integration: load balancers, routes, node lifecycle.	Absent on Minikube

Every node

Component	Responsibility
kubelet	Makes pods real on this node. Watches for pods assigned to it, drives the container runtime, **runs your probes**, reports status. Not a container itself.
kube-proxy	Programs Service routing into the kernel (iptables or IPVS). No proxy process in the request path.
Container runtime (CRI)	containerd — actually runs containers
Network plugin (CNI)	Calico — assigns pod IPs, enforces NetworkPolicy
Storage plugin (CSI)	Attaches and mounts volumes

> **Why this course insists on `--cni=calico`.** Kubernetes does not implement networking; it defines the CNI interface. Minikube's default plugin provides connectivity but **does not enforce NetworkPolicy**. On a default cluster, every Day 4 security policy would apply cleanly and protect nothing — the worst possible outcome in a security module. CNI cannot be changed on a running cluster, which is why L1.1 makes you verify it on Monday.

6. Component interactions

See §1.1 point 6 — the `kubectl apply` trace is the canonical example.

Static pods. On Minikube and kubeadm clusters, the control-plane components themselves run as pods — but they are created by the kubelet directly from files in `/etc/kubernetes/manifests/`, not by the API server. That is the bootstrap answer to "what starts the API server if the API server creates everything?"

7. Enterprise example

A payments processor runs three API servers behind a load balancer across three availability zones, with a five-member etcd cluster on dedicated NVMe. Scheduler and controller-manager run in all three zones with leader election. Losing an entire zone costs one API server and one etcd member; the cluster continues without interruption. Control-plane nodes run *no* workloads — `node-role.kubernetes.io/control-plane:NoSchedule` keeps them clear.

8. Real-world analogy

An airport. The control tower (control plane) does not fly aircraft — it decides which runway each aircraft uses and when. The aircraft (nodes) do the flying. If the tower goes silent, aircraft already airborne continue safely; what stops is new departures and reassignments.

Where it breaks: an airport has one tower. A production cluster has three, any of which can take over instantly.

9. Best practices

Practice	Reason
Three or five etcd members, never even numbers	Raft needs a majority; four members tolerate the same single failure as three but cost more
Put etcd on fast, dedicated disks	etcd is fsync-bound. Disk latency is the number-one cause of control-plane instability.
Do not schedule workloads on control-plane nodes	A noisy pod must never be able to starve the API server
Spread control-plane replicas across failure domains	Otherwise HA is theatre
Enable audit logging	You cannot investigate what you did not record — and regulators will ask
Keep kubectl within one minor version of the server	Version skew produces confusing, silent failures

10. Common mistakes

Mistake	Symptom
Even number of etcd members	No improvement in fault tolerance; cluster loses quorum unexpectedly
etcd on network storage	Random control-plane latency spikes, leader elections, mysterious slowness
Believing the scheduler starts containers	Confusion when debugging `Pending` vs `ContainerCreating`
Ignoring the CNI choice	NetworkPolicies silently not enforced — the failure has no error message
Backing up nothing because "it's declarative"	True only if every manifest is really in Git. Most clusters have drift.

11. Security considerations

Threat	Control	Residual risk
etcd read = every Secret in the cluster, in plaintext	Encryption at rest; restrict etcd to control-plane nodes; mTLS between API server and etcd	Encryption keys must themselves be protected
Compromised API server = full cluster control	Strong authN (OIDC/certs), RBAC, admission control, audit logging	Cluster-admin remains cluster-admin
Kubelet API exposed without auth	`--anonymous-auth=false`, `--authorization-mode=Webhook`	Node compromise still yields local container access
Workloads on control-plane nodes	Taints plus admission policy	Break-glass exceptions

12. Performance considerations

- **API server** scales horizontally; it is stateless. Watch `apiserver_request_duration_seconds` and inflight-request limits.

- **etcd** does not scale horizontally for writes — more members means *more* consensus work. Scale vertically: faster disks.
- **Scheduler throughput** is around 100 pods/second by default. `percentageOfNodesToScore` trades placement quality for speed on large clusters.
- **Watch out for `LIST` storms**: a controller (or a badly written Operator) listing all pods every few seconds is the most common cause of API server saturation.

13. High availability

Component	HA mechanism	Failure behaviour
API server	N replicas behind a load balancer	Stateless; any instance serves
etcd	Raft, 3 or 5 members	Tolerates (N−1)/2 losses; below quorum it goes **read-only**
Scheduler	Leader election	Standby takes over in ~15s
Controller manager	Leader election	Same
kubelet	One per node, not HA	Node marked `NotReady` after ~40s; pods evicted after the toleration period (default 300s)

14. Disaster recovery

Scenario	Recovery
One etcd member lost	Remove, add a replacement, let it sync. No downtime.
etcd quorum lost	Restore from snapshot to a single member, then re-add. **This is the real disaster.** Practise it.
Control plane entirely lost	Rebuild control plane, restore etcd, rejoin nodes. Workloads kept running throughout.
Full cluster lost	Rebuild from manifests in Git. This is why GitOps is a DR strategy, not just a workflow.

RTO/RPO for a practised team: RTO 30–60 minutes, RPO equal to your snapshot interval.

15. Monitoring

Metric	Threshold
`etcd_disk_wal_fsync_duration_seconds` `p99`	> 25 ms — investigate disk
`etcd_server_leader_changes_seen_total`	Any sustained increase
`etcd_server_has_leader`	0 = quorum lost. Page immediately.
`apiserver_request_duration_seconds` `p99`	> 1 s
`apiserver_current_inflight_requests`	Approaching the configured limit
`scheduler_pending_pods`	Sustained > 0 with capacity available
`kubelet_node_ready` (per node)	Any 0

16. Troubleshooting

Symptom	Cause	Command	Fix
`kubectl` times out	API server down / LB misrouting	`curl -k https://<api>:6443/healthz`	Check API server pods and LB
Everything `Pending`, capacity available	Scheduler down	`kubectl -n kube-system get pods -l component=kube-scheduler`	Restart scheduler
Nothing self-heals	controller-manager down	`kubectl -n kube-system logs -l component=kube-controller-manager`	Restart
Node `NotReady`	kubelet down or CNI unhealthy	`kubectl describe node <n>` → Conditions	Check kubelet, CNI pods
Writes fail, reads work	etcd quorum lost	`etcdctl endpoint status --cluster`	Restore quorum
NetworkPolicies do nothing	CNI does not enforce them	`kubectl get pods -n kube-system -l k8s-app=calico-node`	Rebuild with a policy-capable CNI

Interview questions

1. **Name the control-plane components and give each one sentence.** *(junior)*
2. **Does the scheduler start containers?** *No. It writes a node name into the pod object and stops. The kubelet on that node starts the container. Getting this wrong makes `Pending` versus `ContainerCreating` impossible to reason about.*
3. **Why must etcd have an odd number of members?** *Raft requires a majority. Three tolerates one loss; four also tolerates only one — you pay for a member that buys nothing and adds consensus latency.*
4. **How would you design a control plane to survive an availability-zone failure?** *(senior) Three control-plane nodes across three zones, API servers behind a zone-aware load balancer, five etcd members spread so no zone holds a majority, leader election for scheduler and controller-manager, and control-plane nodes tainted against workloads.*

5. **Your NetworkPolicies are applied but traffic still flows. What is wrong?** *(senior) The CNI does not implement policy enforcement. Kubernetes accepts and stores the object regardless — enforcement is the plugin's job. Verify with a policy-capable CNI such as Calico or Cilium.*

1.3 Namespaces

1. What it is

A namespace is a named scope inside a cluster. Object names must be unique within a namespace, not across the cluster.

2. Why it exists

To let many teams, environments and applications share one cluster without colliding — and to give RBAC, quotas and policies something to attach to.

3. The business problem

AxisPay handles cardholder data. A QSA auditing the platform asks one question that determines the cost of the entire audit: **what is in scope?** Everything that stores, processes or transmits cardholder data must meet the full PCI-DSS control set.

A flat, unsegmented platform means everything is in scope — every service, every developer's access, every log. Namespace design is the first line of the segmentation argument.

4. How it works

Most objects are namespaced; some are not. The namespace is part of the object's identity and appears in its DNS name: `payment-service.axispay-core.svc.cluster.local`.

5. Internal architecture

AxisPay's zones:

Namespace	Zone	`pci-scope`	Contains
`axispay-edge`	DMZ	`false`	edge-gateway, auth-service
`axispay-core`	CDE	`true`	payment, merchant, customer, fraud, routing, ledger
`axispay-async`	processing	`true`	settlement, notification, audit, reporting
`axispay-data`	vault	`true`	PostgreSQL, Redis, RabbitMQ *(Day 3)*
`axispay-observability`	monitoring	`false`	Prometheus, Grafana, Loki *(Day 5)*

> `axispay-edge` is `pci-scope: false` because it **transmits** but does not **store or process** cardholder data — and in this platform it only ever sees a token, never a card number. That is a real audit argument and it hinges entirely on the tokenisation boundary.

6. Component interactions

Namespaces are the selector target for:

- **NetworkPolicy** — `namespaceSelector: {matchLabels: {axispay.io/zone: edge}}` *(Day 4)*
- **RBAC** — a `RoleBinding` grants within one namespace *(Day 5)*
- **ResourceQuota / LimitRange** — enforced per namespace *(Day 2)*
- **Pod Security Admission** — enforced by namespace label *(Day 5)*

This is why the labels matter more than the names.

7. Enterprise example

A bank runs one cluster per environment and, within production, one namespace per service domain with a strict naming convention. Namespace creation is automated: creating `payments-prod` automatically provisions a default-deny NetworkPolicy, a ResourceQuota, a LimitRange, RBAC bindings to the owning team's group, and Pod Security labels. Nobody creates a namespace by hand — the guarantees come with it.

8. Real-world analogy

Floors in an office building. Each has its own room numbers, so "Room 3" exists on every floor without confusion, and each floor has its own door badge policy.

Where it breaks: in a building, the walls between floors are physical. In Kubernetes, **namespaces are not walls**. The network between them is wide open by default.

9. Best practices

Practice	Reason
Never use `default` for real workloads	It has no quota, no policy, and no clear owner
Label namespaces with zone, scope, owner and cost centre	Every later control selects on labels, not names
One namespace per trust boundary, not per micro-team	Too many namespaces makes policy unmanageable
Apply ResourceQuota to every namespace	One namespace should never be able to starve another
Automate namespace creation with its guarantees attached	Manual creation always forgets something
Set your context namespace	`kubectl config set-context --current --namespace=<ns>` prevents an entire class of mistake

10. Common mistakes

Mistake	Symptom
Believing namespaces isolate the network	Flat pod network across all namespaces — an audit finding, discovered late
Deploying to `default` by accident	Objects "missing"; quotas and policies not applied
Using `ClusterRoleBinding` where `RoleBinding` was meant	Access granted in **every** namespace, including future ones
Putting environments in namespaces of one cluster	A cluster-wide failure takes production and staging together
Deleting a namespace to "clean up"	Deletes **everything** inside it, including PVCs and their data

11. Security considerations

Threat	Control	Residual risk
Lateral movement between namespaces	NetworkPolicy default-deny *(Day 4)*	Requires a policy-enforcing CNI
Over-broad RBAC	Prefer `Role` over `ClusterRole`; review bindings	Cluster-scoped resources still need ClusterRoles
Privileged pods in a regulated namespace	Pod Security Admission `restricted` *(Day 5)*	Applies at admission only
Secret sprawl	Secrets are namespaced; restrict `get` / `list` per namespace	A namespace admin can read its Secrets

> **A namespace is not a security boundary. It is the handle you hang security boundaries on.**

12. Performance considerations

Namespaces are free at runtime — they add no data-path cost. The cost is control-plane: thousands of namespaces each with quotas, policies and RBAC increases API server memory and watch load. Namespace *deletion* can be slow, because it must delete every contained object and wait for finalisers.

13. High availability

Namespaces are metadata in etcd; they inherit control-plane HA. They do **not** provide availability isolation — a namespace does not confine a failure. Use nodes, zones and PodDisruptionBudgets for that.

14. Disaster recovery

Trivially reproducible from manifests — a namespace is a few lines of YAML. **But deleting one destroys every object inside it, including PersistentVolumeClaims and therefore potentially the data.** Protect production namespaces with RBAC restricting `delete` on namespaces, and set PV reclaim policy to `Retain` for anything holding data.

15. Monitoring

Metric	Why
`kube_namespace_status_phase{phase="Terminating"}`	Namespace stuck terminating — almost always a finaliser
`kube_resourcequota` **used vs hard**	Approaching quota causes confusing scheduling refusals
Object count per namespace	Runaway creation (a bad Operator or CI loop)

16. Troubleshooting

Symptom	Cause	Command	Fix
`namespaces "x" not found`	Not created, or typo	`kubectl get ns`	Apply the manifest
Objects land in `default`	Context not scoped	`kubectl config view --minify | grep namespace`	Set the namespace
Namespace stuck `Terminating`	A finaliser is blocking	`kubectl get ns <n> -o json | jq .spec.finalizers`	Remove the blocking resource first
Pods rejected on create	ResourceQuota exceeded	`kubectl describe quota -n <ns>`	Raise quota or lower requests
Cross-namespace call fails	Used the short DNS name	`nslookup` from inside a pod	Use `<svc>.<ns>.svc.cluster.local`

Interview questions

1. **Do namespaces isolate network traffic?** *No. By default every pod can reach every other pod in any namespace. Namespaces isolate names, RBAC scope and quotas. Network isolation requires NetworkPolicy plus a CNI that enforces it.*
2. **Name three resources that are NOT namespaced.** *Node, PersistentVolume, StorageClass, ClusterRole, ClusterRoleBinding, CustomResourceDefinition, IngressClass, Namespace itself.*
3. **What is the practical difference between RoleBinding and ClusterRoleBinding?** *(senior) A RoleBinding grants within one namespace. A ClusterRoleBinding grants across every namespace, including ones created later. Using the latter by mistake is one of the most common accidental privilege escalations.*
4. **How would you design namespaces for a PCI-regulated payments platform?** *(senior) Segment by trust zone rather than by team: a DMZ holding only the edge, a cardholder data environment, an async processing zone, an isolated data tier, and observability. Label each with zone and PCI scope, then anchor NetworkPolicy, RBAC and Pod Security Admission to those labels. Minimise what is in scope — anything that only ever sees tokens can be argued out of the CDE.*

1.4 Pods

1. What it is

A Pod is one or more containers that are always scheduled together on the same node, share one IP address, and live and die as a unit. It is the smallest thing Kubernetes will schedule.

2. Why it exists

Some containers must share a network namespace and a filesystem to function: an application and its log shipper, an application and a proxy sidecar, an application and an init container that prepares its data. A scheduler that only understood individual containers could not express "these must be co-located".

3. The business problem

AxisPay's `payment-service` needs to run somewhere, with an IP other services can reach, and with the ability to add a log-forwarding sidecar later without rewriting the application.

4. How it works

The kubelet creates a `pause` container first. It does nothing except hold the network and IPC namespaces open. Your containers then *join* those namespaces. That is why your container can crash and restart without the Pod losing its IP address.

Inside a Pod:

- **One IP.** All containers share it and reach each other on `localhost`.
- **One port space.** Two containers cannot both bind 8080.
- **Shared volumes.** Any container can mount them.
- **Separate filesystems** (except mounted volumes) and separate process spaces by default.

5. Internal architecture

Container types within a Pod:

Type	Runs	Purpose
`initContainers`	Sequentially, to completion, before app containers	Wait for a dependency, run a migration, fetch config *(Day 3)*
`containers`	Concurrently, for the Pod's life	Your application
Sidecar (`initContainer` with `restartPolicy: Always`)	Starts before app containers, runs alongside	Log shipper, proxy *(Day 5)*

Pod phases:

Phase	Meaning
`Pending`	Accepted, not yet running — being scheduled, image pulling, or volume mounting
`Running`	Bound to a node; at least one container running
`Succeeded`	All containers exited 0 and will not restart
`Failed`	All terminated; at least one failed
`Unknown`	Node unreachable

> **`Running` does not mean `Ready`.** `Running` says the container process started. `Ready` says a readiness probe passed and the Pod may receive traffic. `kubectl get pods` shows `1/1` or `0/1` for exactly this reason, and the difference is where most Day 2 confusion lives.

6. Component interactions

```
scheduler  writes spec.nodeName
kubelet    sees a pod assigned to it
kubelet    CRI: create pause container  → network namespace exists
kubelet    CNI: assign an IP to that namespace
kubelet    CSI: mount any volumes
kubelet    CRI: run initContainers, in order, to completion
kubelet    CRI: start app containers
kubelet    begins probing; writes status back to the API server
```

7. Enterprise example

A payment service Pod in production runs three containers: an init container that waits for the database and applies migrations; the application; and a sidecar that forwards logs and exposes metrics. All three share the Pod's IP and a volume for temporary files. The application does not know the sidecar exists — a deliberate separation that lets the platform team change log shipping without a single application release.

8. Real-world analogy

A shipping container on a ship. Everything inside travels together, arrives together and is unloaded together. You do not ship half a container to Durban and half to Cape Town.

Where it breaks: a shipping container is sealed and independent. Pods share a network identity with the cluster and are constantly created and destroyed — a container that is thrown away and rebuilt whenever convenient.

9. Best practices

Practice	Reason
Never run a bare Pod in production	Nothing recreates it. Use a Deployment, StatefulSet or Job.
One concern per container	You cannot scale, restart or resource-limit two concerns independently
Log JSON to stdout	The kubelet captures stdout. A log file inside a container is unreadable and vanishes on restart.
Run as non-root in the image and enforce it in the pod spec	Defence in depth: the image is correct, and the cluster verifies it
Pin image tags; never `:latest`	Otherwise what is running becomes unknowable and rollbacks stop working
Use the Downward API for pod identity	Enables per-pod diagnostics such as `/api/v1/_info`
Set `terminationGracePeriodSeconds` deliberately	Default 30s; payment workloads often need longer to drain

10. Common mistakes

Mistake	Symptom
Bare Pod in production	It dies and stays dead
Logging to a file inside the container	`kubectl logs` empty; logs lost on restart
Running as root	Container escape becomes host compromise
Expecting logs after a crash	`container is waiting to start` — use `--previous`
Using `:latest`	Nobody can say what is deployed
Two unrelated processes in one container	Cannot scale or restart them independently; no supervision
Assuming `Running` means healthy	Traffic sent to a pod that cannot serve

11. Security considerations

Threat	Control	Residual risk
Container escape via root	`runAsNonRoot`, `runAsUser: 10001`, drop all capabilities	Kernel vulnerabilities remain
Writable filesystem enables persistence	`readOnlyRootFilesystem: true` plus an `emptyDir` for scratch	Application must tolerate it
Privilege escalation via setuid	`allowPrivilegeEscalation: false`	—
hostPath mounts expose the node	Forbid via Pod Security Admission	Some system workloads legitimately need it
Secrets visible in `describe`	Use Secret volumes, not env vars; restrict RBAC	Secrets are base64, not encrypted *(see Day 3)*

12. Performance considerations

- **Startup time** is dominated by image pull. Keep images small; prefer multi-stage builds; pre-pull on nodes.
- **Init containers are serial** and add directly to startup latency. A slow `wait-for-db` delays every rollout.
- **Sidecars share the Pod's resource budget.** A log shipper with no limits can starve the application.
- **Pods per node** default to 110. On small nodes the practical limit is memory, not this number.

13. High availability

A single Pod has none. HA comes from running several via a controller, and from spreading them across nodes and zones with anti-affinity and topology spread constraints *(Day 4)*. A Pod is bound to one node for its entire life — it is never migrated. "Rescheduling" means **deleting** it and **creating a new one** elsewhere.

14. Disaster recovery

Pods are disposable and hold no state worth recovering — unless they write to a volume, in which case the volume is what matters *(Day 3)*. The recovery unit is the controller's manifest, not the Pod.

15. Monitoring

Metric	Threshold
`kube_pod_status_phase{phase="Pending"}`	Sustained > 2 min
`kube_pod_container_status_restarts_total` rate	Any sustained increase
`kube_pod_container_status_last_terminated_reason{reason="OOMKilled"}`	Any
`kube_pod_status_ready`	0 while `Running`
`container_start_time_seconds`	Startup regression after a release

16. Troubleshooting

Symptom	Cause	Command	Fix
`Pending`	No node fits; PVC unbound; taints	`kubectl describe pod` → Events	Add capacity, fix claim, tolerate taint
`ImagePullBackOff`	Bad tag, missing image, no credentials	`kubectl describe pod` → Events. **Logs are useless.**	Fix tag or build the image
`CrashLoopBackOff`	App exits repeatedly	`kubectl logs <pod> --previous`	Read the traceback
`OOMKilled`, exit 137	Exceeded memory limit	`kubectl describe pod` → Last State	Raise the limit or fix the leak
`Running` but `0/1`	Readiness probe failing	`kubectl describe pod` → Conditions	Check the dependency
`Terminating` forever	Finaliser or long grace period	`kubectl get pod -o json | jq .metadata.finalizers`	Resolve the finaliser
`kubectl exec` fails	Multiple containers; no shell	`kubectl get pod -o jsonpath='{.spec.containers[*].name}'`	Use `-c <name>`; or `kubectl debug`

Interview questions

1. **Why does the Pod exist rather than scheduling containers directly?** *Because some containers must share a network namespace and filesystem to function — app plus sidecar, app plus init container. The Pod expresses "these must be co-located, share one IP, and live and die together".*
2. **What does the `pause` container do?** *It holds the Pod's network and IPC namespaces open so application containers can join them, and can restart independently without the Pod losing its IP.*
3. **A pod is `Running` but shows `0/1`. What does that mean?** *The container process started but the readiness probe is failing, so the Pod has been removed from Service endpoints. It is alive but not receiving traffic — usually a dependency it needs is unavailable.*
4. **`kubectl logs` returns "container is waiting to start". What now?** *(senior) There is no container yet, so there are no logs — this is an `ImagePullBackOff` or a volume-mount failure. Go to `kubectl describe pod` and read the Events, not the logs.*
5. **How would you design a Pod for a service that must drain in-flight payments before terminating?** *(senior) A `preStop` hook that marks the pod unready and sleeps long enough for endpoint propagation, an application that handles SIGTERM by refusing new work while finishing in-flight requests, and `terminationGracePeriodSeconds` set comfortably above the longest expected request. Readiness must fail before the process stops accepting connections, or traffic is severed mid-authorisation.*

1.5 Deployments and ReplicaSets

1. What it is

A Deployment is a controller that keeps a stated number of identical Pods running, and manages the transition when you change them.

2. Why it exists

Bare Pods do not recover. Deployments add self-healing, scaling, and — crucially — a safe way to change what is running without downtime.

3. The business problem

The 02:14 page from §1.1. Also: AxisPay must deploy a new fraud model on a Tuesday afternoon, during trading, without dropping a single authorisation.

4. How it works

```
Deployment  ──owns──▶  ReplicaSet  ──owns──▶  Pods
```

A Deployment never creates a Pod. It creates and manages ReplicaSets; a ReplicaSet creates Pods.

This is not trivia. It is the mechanism behind every rolling update: the Deployment creates a **new** ReplicaSet and gradually shifts replicas from the old one to the new. Both continue to exist. **A rollback is simply scaling the old ReplicaSet back up** — no image pull, no new object, near-instant.

5. Internal architecture

Field	Meaning
`spec.replicas`	Desired count
`spec.selector.matchLabels`	Which pods this Deployment owns. **Immutable.**
`spec.template`	The Pod template. Changing it triggers a new ReplicaSet.
`spec.strategy`	`RollingUpdate` (default) or `Recreate`
`spec.revisionHistoryLimit`	Old ReplicaSets kept for rollback (default 10)

The ReplicaSet name is `<deployment>-<pod-template-hash>`. The hash is computed from `spec.template`, which is why changing the image creates a new ReplicaSet but changing `spec.replicas` does not.

6. Component interactions

```
you             kubectl apply (new image)
Deployment ctrl computes a new pod-template-hash → creates ReplicaSet B
Deployment ctrl scales B up by maxSurge
ReplicaSet B    creates pods
kubelet         starts containers; readiness probes begin passing
Deployment ctrl sees new pods Ready → scales ReplicaSet A down
                repeats until B = desired and A = 0
                ReplicaSet A is RETAINED at 0 replicas, for rollback
```

7. Enterprise example

A payments platform deploys 40 times a day with `maxUnavailable: 0` and `maxSurge: 1`, so capacity never drops below 100% during a release. Every Deployment has a PodDisruptionBudget and anti-affinity across zones. A failed readiness probe halts the rollout automatically, leaving the previous version serving — the release fails safe, without a human deciding anything.

8. Real-world analogy

A shop manager told "there must always be three staff on the floor". Someone calls in sick; the manager calls in a replacement. That is the ReplicaSet.

Now the manager is told "swap all three for the new uniform, without the shop ever being understaffed" — bring in one new-uniform staff member, send one old-uniform member home, repeat. That is the Deployment.

Where it breaks: the manager reacts to events. The controller continuously compares the roster with the floor, regardless of what happened.

9. Best practices

Practice	Reason
Put only never-changing labels in `spec.selector`	It is immutable; fixing it means deleting and recreating the Deployment
Set `maxUnavailable: 0` for user-facing services	Capacity never drops during a release
Always define readiness probes	Without them a rollout "succeeds" while dropping traffic *(Day 2)*
Set `revisionHistoryLimit` explicitly	Default 10 ReplicaSets per Deployment adds up across a platform
Record a change cause	`kubectl rollout history` becomes readable during an incident
Use `kubectl rollout status` in CI	Makes the pipeline wait for, and fail on, a bad release
Prefer `Recreate` only for singletons that cannot run two copies	E.g. a schema migration job

10. Common mistakes

Mistake	Symptom
Version number in `spec.selector`	Every release orphans the previous pods
Selector not a subset of template labels	Rejected at admission — the message is not obvious
`kubectl scale` without updating YAML	Silent revert on next apply
No readiness probe	Rollout completes; traffic is dropped
Relabelling a running pod	It leaves both its Service and its ReplicaSet; a replacement appears; you now have an orphan
Expecting the Deployment to own Pods directly	Cannot explain rollback; confused by ReplicaSets in `get all`
Deleting a ReplicaSet by hand	The Deployment recreates it immediately

11. Security considerations

Threat	Control	Residual risk
Anyone with `update` on Deployments can run arbitrary images	RBAC per namespace; admission control on registries	CI credentials remain powerful
A rollback re-introduces a vulnerable image	Scan on pull; block known-bad digests at admission	Rollback during an incident may bypass policy
Deployment mounting a Secret it should not	RBAC on Secrets; admission policy	Namespace admins can read their Secrets

12. Performance considerations

- **Rollout speed** is bounded by `maxSurge`, readiness-probe timing and image pull. `initialDelaySeconds` set too high makes every release slow.
- **`maxUnavailable: 0` requires spare capacity** — the cluster must fit `replicas + maxSurge` pods during the rollout.
- **Large replica counts** amplify everything: 200 replicas at `maxSurge: 1` takes a very long time. Use a percentage.
- **Old ReplicaSets** consume etcd space and API server memory. Keep `revisionHistoryLimit` modest.

13. High availability

Replicas alone are not HA — three replicas on one node all die with that node. Real availability needs:

- Replicas ≥ 3 across ≥ 3 nodes
- Pod anti-affinity or topology spread constraints *(Day 4)*
- PodDisruptionBudget so voluntary disruptions (drains, upgrades) cannot take them all *(Day 4)*
- Readiness probes so traffic only reaches pods that can serve *(Day 2)*

14. Disaster recovery

The Deployment manifest **is** the recovery artefact. Keep it in Git and a lost Deployment is one `kubectl apply` away. Rollback within the cluster is `kubectl rollout undo`, limited by `revisionHistoryLimit`. Beyond that horizon, recovery means re-applying an older manifest from Git — another reason the repository, not the cluster, should be the source of truth.

15. Monitoring

Metric	Threshold
`kube_deployment_status_replicas_available` vs `kube_deployment_spec_replicas`	Gap > 5 min — **the single most useful deployment alert**
`kube_deployment_status_observed_generation` vs `metadata.generation`	Lagging = controller not reconciling
`kube_replicaset_status_replicas` per RS	Two RSs both non-zero for a long time = stuck rollout
Restart rate across the Deployment	Rising after a release = bad build

> The alert that would have caught Day 1's incident before a merchant phoned: `promql kube_deployment_status_replicas_available{deployment="payment-service"} < kube_deployment_spec_replicas{deployment="payment-service"}` for 5 minutes. You will deploy exactly this on Day 5.

16. Troubleshooting

Symptom	Cause	Command	Fix
Rollout stuck, never completes	New pods never Ready	`kubectl rollout status`; `kubectl describe pod`	Fix the probe or the app
`ProgressDeadlineExceeded`	Rollout exceeded `progressDeadlineSeconds` (default 600)	`kubectl describe deploy`	Investigate the new pods
Deleted pod not replaced	You deleted the Deployment, not a Pod	`kubectl get deploy`	Re-apply
Replica count keeps changing on its own	HPA active, or overlapping selectors	`kubectl get hpa`; compare selectors	Expected, or make selectors disjoint
Rollback does nothing	History pruned by `revisionHistoryLimit`	`kubectl rollout history`	Re-apply an older manifest from Git
More pods than `replicas`	An orphaned pod from a relabel	`kubectl get pods --show-labels`	Delete the orphan

Interview questions

1. **What owns a Pod created by a Deployment?** *The ReplicaSet. The Deployment owns the ReplicaSet; the ReplicaSet owns the Pods. This is the mechanism behind rolling updates and rollbacks.*
2. **Why does Kubernetes keep old ReplicaSets at zero replicas?** *So a rollback is just scaling one back up — no image pull, no new object, near-instant.* `revisionHistoryLimit` *controls how many are kept.*
3. **Why is** `spec.selector` **immutable?** *Because changing which pods a controller owns mid-flight would orphan or double-own running pods. Kubernetes forbids it; fixing a selector requires deleting and recreating the Deployment — in production, with traffic on it.*
4. **You set** `maxUnavailable: 0` **and** `maxSurge: 1` **with 2 replicas. Walk through the rollout.** *(senior) One new pod is created (3 total). When it becomes Ready, one old pod is terminated (2 total). Repeat. Capacity never drops below 2. It requires headroom for one extra pod and it is slower than a permissive strategy — the correct trade for a payment path.*
5. **A rollout is stuck at 2/3 updated. How do you diagnose it?** *(senior)* `kubectl rollout status` *for the stall, then* `kubectl get pods` *to find the pod that is not Ready, then* `describe` *for probe failures and events, then* `logs` *for the application's own account. Most often it is a readiness probe failing against an unavailable dependency, or a resource limit that the new version exceeds.*

1.6 Services

1. What it is

A Service is a stable virtual IP and DNS name that load-balances to a continuously updated set of Pods, selected by label.

2. Why it exists

Self-healing makes Pods disposable — and their IP addresses go with them. Something must hold a stable address on their behalf.

3. The business problem

A junior engineer hard-coded a pod IP into the gateway configuration. It worked for four hours. Then the pod was rescheduled, got a new IP, and every card authorisation failed until someone noticed.

4. How it works

```
Service (selector) ⟶ EndpointSlice ⟶ kube-proxy ⟶ kernel rules ⟶ Pods
```

1. You define a Service with a **label selector**.
2. The **endpoint controller** continuously evaluates that selector and writes matching, *ready* pod IPs into an **EndpointSlice**.
3. **kube-proxy** on every node watches EndpointSlices and programs iptables or IPVS rules.
4. Traffic to the ClusterIP is rewritten by the **kernel** to one of the pod IPs.

> **The direction of causation is what people get wrong.** A Service does not *contain* pods. It *selects* them, continuously. Change a pod's labels and it silently leaves the Service. Nothing errors.

There is **no proxy process in the request path** — it is kernel rules. That is why a ClusterIP Service adds almost no latency.

5. Internal architecture

Type	Behaviour	Use
`ClusterIP`	Virtual IP reachable only inside the cluster (default)	Service-to-service — all of Day 1
`NodePort`	Opens the same port on every node	Development; behind an external LB
`LoadBalancer`	Provisions a cloud load balancer	Production external exposure
`ExternalName`	Returns a CNAME; no proxying	Aliasing an external dependency
Headless (`clusterIP: None`)	No virtual IP; DNS returns pod IPs	StatefulSets, client-side balancing *(Day 3)*

DNS forms (mechanism covered on Day 4):

Form	Resolves from
`payment-service`	Same namespace only
`payment-service.axispay-core`	Anywhere
`payment-service.axispay-core.svc.cluster.local`	Anywhere — fully qualified

6. Component interactions

```
you               create Service with selector
endpoint ctrl     lists pods matching the selector AND Ready
                  writes their IPs into an EndpointSlice
kube-proxy (all)  watches EndpointSlices → programs iptables/IPVS
CoreDNS           watches Services → answers <svc>.<ns>.svc.cluster.local
client pod        resolves the name → gets ClusterIP
                  connects → kernel DNATs to a pod IP
```

Only `Ready` pods are included. This is the direct link between readiness probes and traffic routing, and it is why a readiness probe is the mechanism behind zero-downtime deployments.

7. Enterprise example

A payments platform exposes every internal service as ClusterIP and admits external traffic only through an Ingress in a DMZ namespace. Internal calls use fully qualified DNS names supplied by ConfigMap, so the same manifests promote unchanged from dev to production. Session affinity is deliberately **off** — payment requests are stateless and must be free to land anywhere.

8. Real-world analogy

A company switchboard number. Staff come and go and change desks; the switchboard number never changes and always connects you to whoever is currently on duty.

Where it breaks: a switchboard operator is a process in the middle. A Kubernetes Service is kernel rules — after the connection is established there is nothing in the path at all.

9. Best practices

Practice	Reason
Use named `targetPort`	The container can change its port without the Service changing
Keep selectors minimal and stable	Fewer labels, fewer ways to break silently
ClusterIP by default; expose only at the edge	Every NodePort is an attack surface
Use FQDNs in configuration	Removes an entire class of cross-namespace bug
Check EndpointSlices, not just the Service	A Service always has an IP; that proves nothing
Avoid session affinity unless genuinely required	It defeats load balancing and complicates rollouts
Remember load balancing is per connection	HTTP keep-alive and gRPC pin to one pod

10. Common mistakes

Mistake	Symptom
Selector does not match pod labels	Service exists, has an IP, has **no endpoints**. Nothing errors.
`targetPort` wrong	Endpoints exist; connections refused
Short DNS name across namespaces	Name resolution failure
Assuming per-request load balancing	One pod gets all traffic over a keep-alive connection
Expecting `kubectl get svc` to reveal the problem	It never shows endpoint health
Exposing internal services with NodePort	Unnecessary external exposure

11. Security considerations

Threat	Control	Residual risk
Any pod can reach any Service by default	NetworkPolicy default-deny *(Day 4)*	Requires a policy-enforcing CNI
NodePort exposes a service on every node	Prefer Ingress; restrict with firewall rules	Node IPs may be reachable internally
Service name enumeration reveals architecture	RBAC on `list services`; NetworkPolicy	DNS is broadly readable in-cluster
Traffic between pods is plaintext	mTLS via a service mesh (beyond this course)	Meshes add significant operational cost

12. Performance considerations

- **iptables mode** is O(n) in rule evaluation; it degrades noticeably beyond a few thousand Services. **IPVS mode** uses hash tables and scales much better.
- **EndpointSlices** (replacing the older Endpoints object) exist precisely to avoid shipping one enormous object to every node on every pod change.

- **DNS is often the real latency cost.** The default `ndots:5` means short names generate several failed lookups before the right one. Using FQDNs avoids it *(Day 4)*.
- `externalTrafficPolicy: Local` preserves the client source IP and avoids an extra hop, at the cost of uneven balancing.

13. High availability

Services are highly available by construction: the ClusterIP is a kernel rule present on **every** node, so there is no single point of failure and nothing to fail over. If kube-proxy dies on a node, existing connections survive; only rule *updates* stop.

Availability depends on having ready endpoints — which depends on replicas, spread and readiness probes.

14. Disaster recovery

Services are pure configuration and trivially reproducible from manifests. The one caveat: a `LoadBalancer` Service usually holds an allocated external IP. Recreating it may allocate a **different** IP, which breaks DNS and any allowlists your merchants maintain. In production, reserve static IPs and reference them explicitly.

15. Monitoring

Metric	Threshold
`kube_endpoint_address_available`	0 while the Service exists — **page**
`kube_endpoint_address_not_ready`	Sustained > 0
Endpoint count vs replica count	Persistent mismatch
`kubeproxy_sync_proxy_rules_duration_seconds`	Rising = too many Services/endpoints
CoreDNS request/error rate	Errors indicate DNS problems before services notice

16. Troubleshooting

Symptom	Cause	Command	Fix
No endpoints	Selector mismatch, or no pod is Ready	`kubectl get endpointslice -n <ns> -l kubernetes.io/service-name=<svc>`	Compare `svc.spec.selector` with `pod.metadata.labels`
Endpoints exist, connection refused	Wrong `targetPort`	`kubectl get svc <s> -o yaml`	Match the container port
Works in-namespace, fails across	Short name used	`nslookup` from inside a pod	Use the FQDN
One pod gets all traffic	Client keep-alive	Test with separate connections	Expected — not a Service bug
Intermittent failures	Some pods not Ready	`kubectl get pods -o wide`	Fix readiness
Nothing resolves at all	CoreDNS unhealthy	`kubectl -n kube-system get pods -l k8s-app=kube-dns`	Restart CoreDNS

Interview questions

1. **A Service has a ClusterIP but requests fail. What is the first thing you check?** *The EndpointSlice. A Service always has an IP whether or not its selector matches anything, so `kubectl get svc` looks healthy either way. "No endpoints" is the single most common Service bug.*
2. **How does a Service know which pods to send traffic to?** *It does not "know" — it selects. The endpoint controller continuously evaluates the label selector and writes matching, ready pod IPs into an EndpointSlice. kube-proxy programs the kernel from that.*
3. **Is there a proxy process in the request path?** *No. kube-proxy programs iptables or IPVS rules; the kernel does the rewriting. That is why ClusterIP adds almost no latency.*
4. **Why does gRPC need special handling behind a Kubernetes Service?** *(senior) Because kube-proxy load-balances connections, not requests. gRPC multiplexes many requests over one long-lived HTTP/2 connection, so all of them pin to a single pod. Solutions are client-side load balancing with a headless Service, a service mesh, or a proxy that understands HTTP/2.*
5. **You relabel a running pod. What happens to the Service and the ReplicaSet?** *(senior) It leaves both. The endpoint controller drops it from the EndpointSlice, so it stops receiving traffic. The ReplicaSet no longer counts it, observes a shortfall, and creates a replacement. You end up with an orphaned pod that nothing manages and that will survive deleting the Deployment.*

Day 1 cheat sheet

The 6-step triage loop

```
1. DESIRED state?     kubectl get <kind> <name> -o yaml
2. ACTUAL state?      kubectl get pods -o wide
3. CLUSTER says?      kubectl describe <kind> <name>
                      kubectl get events --sort-by='.lastTimestamp' | tail -20
4. APP says?          kubectl logs <pod> [--previous] [-c <container>]
5. From INSIDE?       kubectl exec -it <pod> -- sh
                      kubectl debug -it <pod> --image=busybox
                      kubectl port-forward <pod> 8080:8080
6. What CHANGED?      kubectl rollout history deployment/<name>
                      git log --oneline -10
   ____________________

   FIX → VERIFY → what would have caught this first?
```

Pod status decision table

Status	Container started?	First move
`Pending`	No — not even placed	`describe` → Events
`ContainerCreating`	No — being set up	`describe` → image pull or volume
`ImagePullBackOff`	**Never**	`describe` → Events. Logs are useless.
`CrashLoopBackOff`	Yes, then exited	`logs --previous`
`Running` `0/1`	Yes, not serving	Readiness probe / dependency
`Running` `1/1`	Yes, healthy	Look higher: Service, DNS, Ingress
`Terminating` (stuck)	Shutting down	Finaliser or grace period

Commands you will use every day

```
# context and scope
kubectl config use-context axispay
kubectl config set-context --current --namespace=axispay-core

# see everything AxisPay
kubectl get all -A -l app.kubernetes.io/part-of=axispay
kubectl get pods -A -o wide

# what did day 3 add?
kubectl get all -A -l axispay.io/day-introduced=3

# the four that solve most problems
kubectl describe pod <pod>
kubectl logs <pod> --previous
kubectl get events --sort-by='.lastTimestamp' | tail -20
kubectl get endpointslice -n <ns> -l kubernetes.io/service-name=<svc>

# built-in documentation — offline
kubectl explain deployment.spec.strategy
kubectl api-resources --namespaced=false

# reach a pod from your laptop
kubectl port-forward -n axispay-core deploy/payment-service 8083:8080

# throwaway debug pod inside the cluster
kubectl run dbg -n axispay-edge --rm -it --restart=Never \
  --image=curlimages/curl:8.11.1 -- sh
```

AxisPay conventions

Thing	Format	Example
Merchant ID	`MER_` + 10 chars	`MER_7QK2XD9P4A`
Payment ID	`pay_` + 24 hex	`pay_9f2c41ab77de0c3518be4d6a`
Reference	`AXP-YYYYMMDD-` + 8 hex	`AXP-20260810-4c9a1f77`
Card token	`tok_` + 24 hex	`tok_a71ef4c2900bd5386ff1240e`
Money	Integer **minor units** + ISO-4217	`129900` `ZAR` = R1,299.00

Every service exposes: `/healthz` · `/readyz` · `/startupz` · `/metrics` · `/api/v1/_info`

Day 1 review questions

1. What are the two states every Kubernetes object has, and who writes each?
2. Trace what happens when you delete one pod of a 3-replica Deployment.
3. Name every control-plane component and give each a one-sentence responsibility.
4. Does the scheduler start containers?
5. Do namespaces isolate network traffic? Justify your answer.
6. Name three cluster-scoped resources.
7. Why does the Pod exist rather than scheduling containers directly?
8. What does the `pause` container do?
9. A pod is `Running` but `0/1`. What does that mean, and what is your first command?
10. What owns a Pod created by a Deployment, and why does that matter?
11. Why is `spec.selector` immutable?
12. Why does Kubernetes retain old ReplicaSets at zero replicas?
13. A Service has a ClusterIP but requests fail. What is your first command?
14. How does a Service decide which pods receive traffic?
15. Why is there no proxy process in a ClusterIP request path?
16. `kubectl logs` says "container is waiting to start". What is happening and what do you do?
17. How do you distinguish `ImagePullBackOff` from `CrashLoopBackOff` from the output of one command?
18. What single idea underlies every controller in Kubernetes?

Answers: `documents/assessments/answer-keys/day1-answer-key.md`

Day 1 summary

You built: three labelled namespaces · four Deployments · four Services · nine pods across two namespaces · one real payment processed end-to-end with correct fee arithmetic and working idempotency.

You learned: the declarative model and the reconciliation loop · cluster architecture and the hub-and-spoke communication pattern · namespaces as trust boundaries · Pods and why bare ones are unfit for production · Deployments, ReplicaSets and self-healing · Services, label selection and load balancing · a repeatable triage method, applied under pressure to a real incident.

What is still missing — and when you fix it:

Gap	Consequence	Fixed
No resource requests	The scheduler is guessing; pods land badly and get OOM-killed	L2.1
No probes	Kubernetes cannot tell "started" from "able to serve" — today's incident took out all three replicas	L2.3
No autoscaling	Black Friday is a manual `kubectl scale` at 2am	L2.4
No graceful shutdown	A rolling update severs in-flight payments	L2.6
Nothing scheduled	Settlement at 23:00 is a cron job on someone's laptop	L2.5
Payments vanish on restart	All state is in memory	Day 3
Config and signing key in plain env vars	Visible to anyone with namespace read access	Day 3
Unreachable from outside the cluster	No merchant can actually integrate	Day 4
Edge can reach the data tier	A PCI finding	Day 4
No RBAC, metrics, dashboards or alerts	You cannot operate what you cannot see	Day 5

Tonight (optional, 20 minutes): run `kubectl explain deployment.spec.strategy` and read it. Tomorrow starts there.